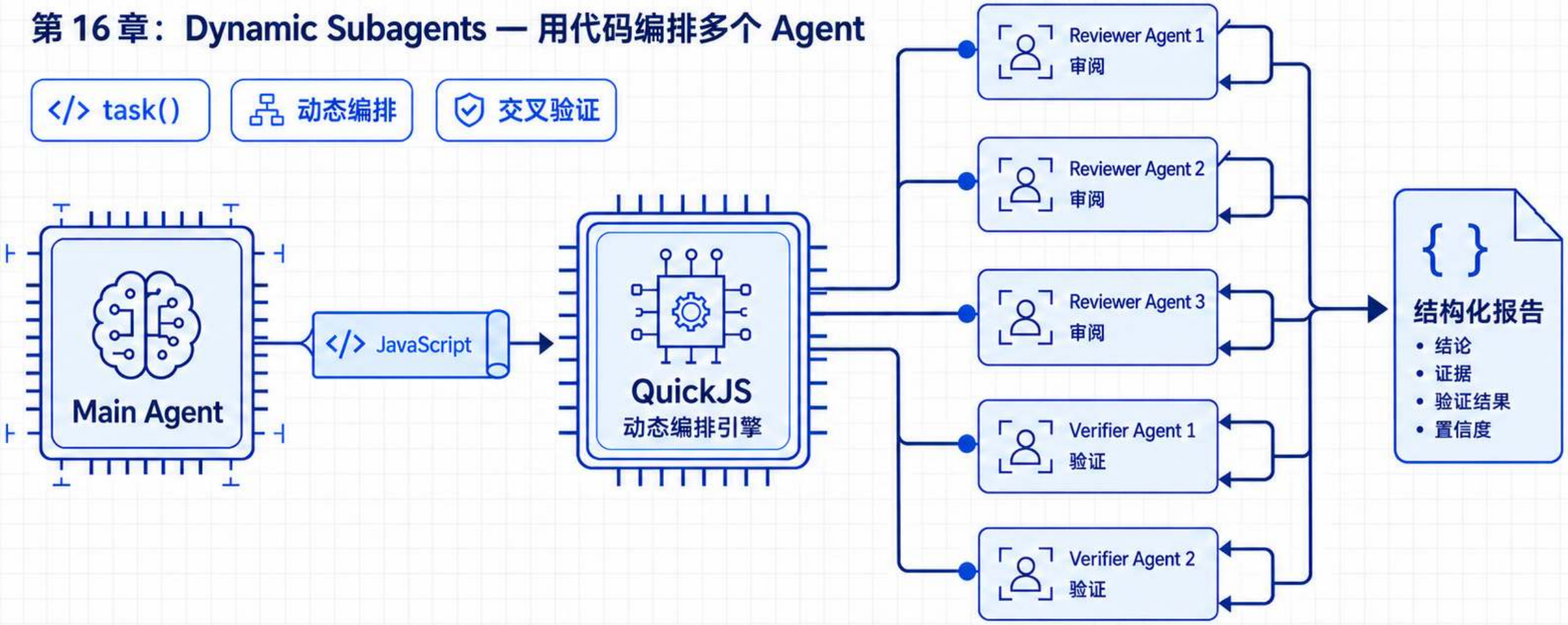
Deep Agents 实战

第 16 章：Dynamic Subagents — 用代码编排多个 Agent

`</> task()` 动态编排 交叉验证



Beta API; Python 3.11+; 章节参考 deepagents 0.7.8 langchain-quickjs 0.3.5

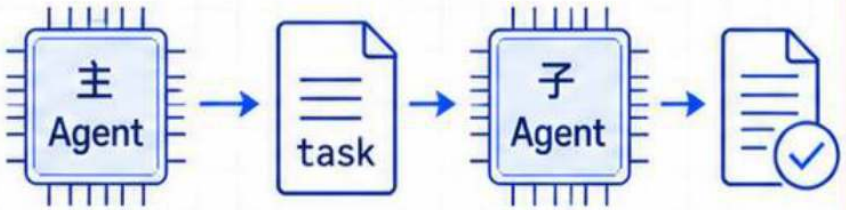
⚠ 代码表达覆盖规则，不保证子任务成功或结论正确。

单次委派，还是代码编排？

一次明确交接用普通 task Tool；批量或多阶段流程再考虑代码编排

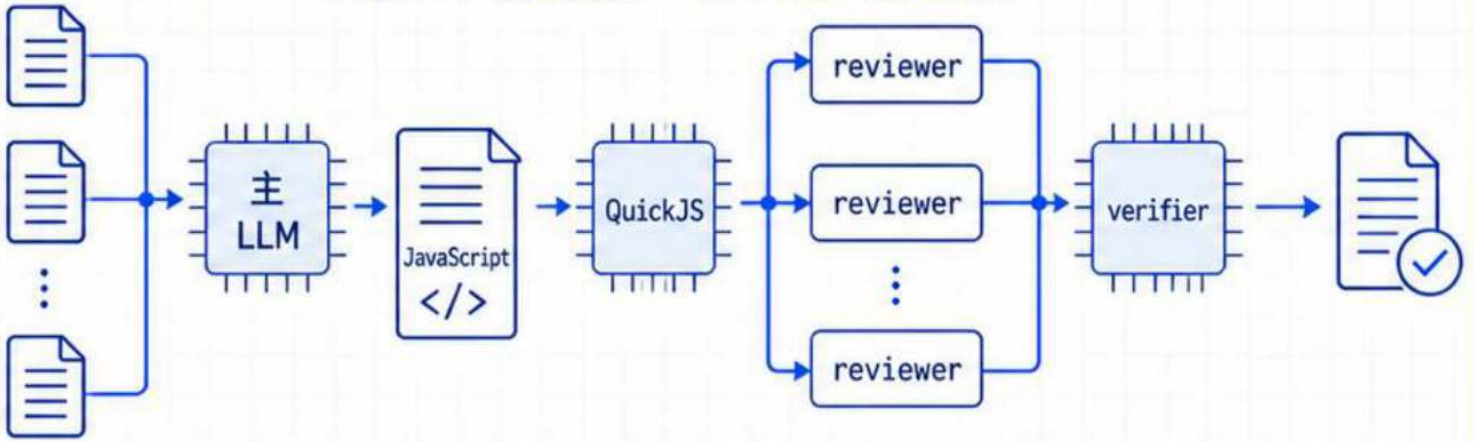
普通 task Tool

一个明确任务 → 一个子 Agent



代码编排

完整输入 → 主 LLM → JavaScript → QuickJS
reviewer 覆盖输入 → verifier 独立复核



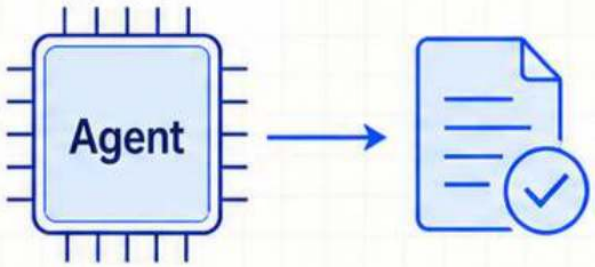
⚠ 生成代码可以表达覆盖规则与阶段关系，但不保证每次调度或判断都正确

先看任务形状，再选择子 Agent 机制

单次委派、批量编排和后台生命周期不是一回事

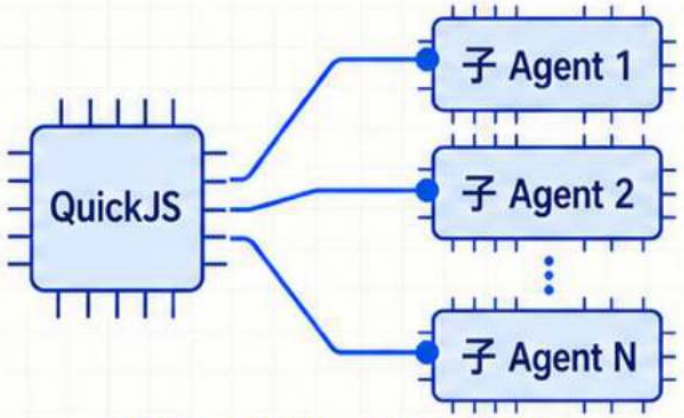
任务怎样展开？

单个明确任务 → 普通 task Tool



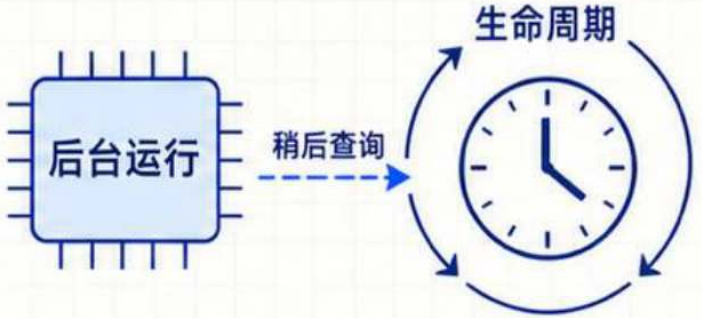
专业 Agent（单一专长）

批量 / 多阶段 → Dynamic Subagents



并行 / 条件 / 顺序编排

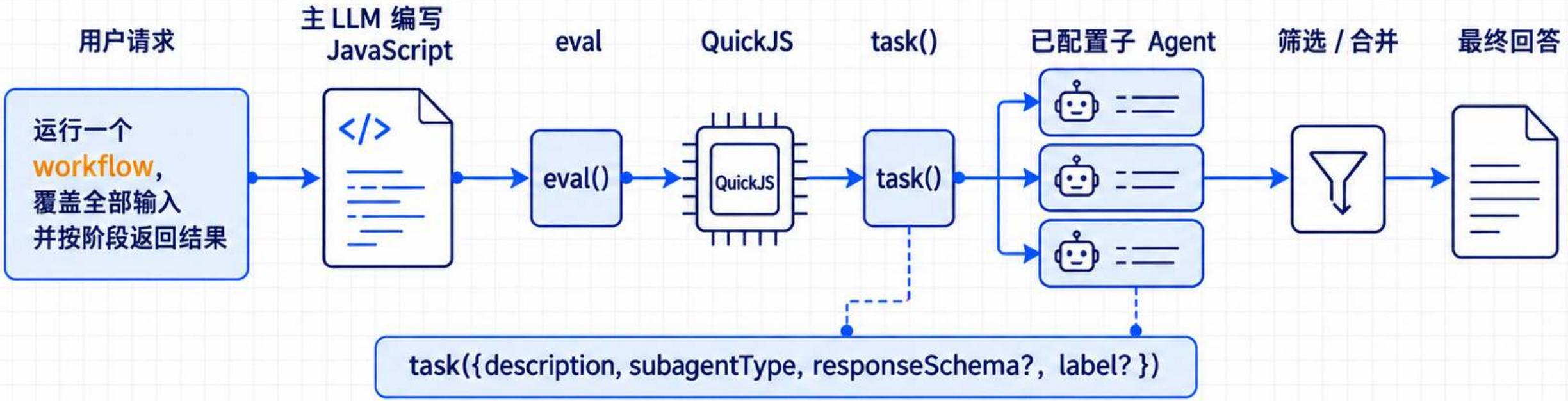
后台持续运行 → Async Subagents



异步执行 + 生命周期管理

JavaScript 是怎样跑起来的

workflow 是提示信号，不是 API 开关



每次调度运行完整 Agent 循环；

description 必须自包含。

安全边界：eval 内部 task 不逐次触发主 Agent 的 task 审批；高影响动作仍需在执行处授权。

task()：两个必填字段，两个可选字段

description | 必填
自包含任务说明

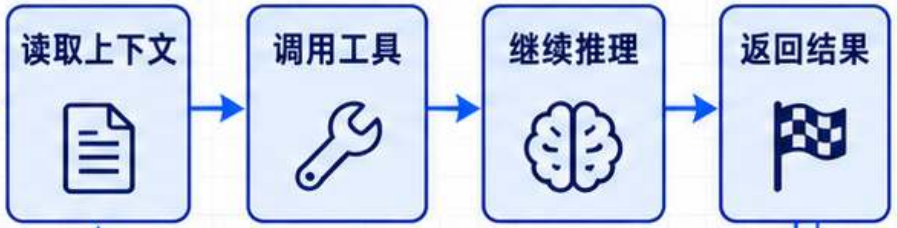
subagentType | 必填
匹配已配置 name

responseSchema | 可选
声明式子 Agent 的结构化返回

label | 可选
仅 UI 标签，不传入任务提示

task()

```
const review = await task({
  description: "审查 src/auth.ts，引用证据",
  subagentType: "reviewer",
  responseSchema: findingsSchema,
  label: "认证审查"
});
```



有 Schema → JavaScript 对象

无 Schema → 本次运行结果

findingsSchema 需自行定义；返回对象无需 JSON.parse。

一次调用返回一次结果，不是可继续对话的会话。

⚠ 0.3.5 源码含 label；runnable-backed 子 Agent 不接受动态 Schema。

先准备角色，再描述流程

配置好的角色决定“能调度谁”，流程提示决定“怎样调度”

开发者先准备

- name / description / system_prompt
- subagents 配置
- CodeInterpreterMiddleware(subagents=True)
- classifier / bug-fixer
- reviewer / verifier

用户再描述

- workflow
- 输入范围
- 角色顺序
- 结果要求
- 停止条件

主 LLM

为当前请求生成 JavaScript

角色配置 + 流程提示 → 一次具体编排

⚠ Beta | Python 3.11+ | 章节参考 deepagents 0.7.8 / langchain-quickjs 0.3.5 | ⚠ 复现实验固定完整依赖。

用 AgentSeek Template 跑通 Dynamic Subagents

deepagents/subagents-dynamic 把六种编排场景、运行证据与生命周期一起打包

安装与创建

01 安装 AgentSeek

```
uv tool install agentseek
```

02 创建应用模板

```
agentseek create deepagents/subagents-dynamic --checkout main --no-input
```

```
cd dynamic_subagents_lab
```

配置与启动

03 准备环境

```
cp .env.example .env
cp frontend/.env.example frontend/.env
编辑 .env：模型、Base URL 与 API Key
```

04 安装依赖并启动

```
agentseek task sync
agentseek task frontend
agentseek dev
```

启动后得到



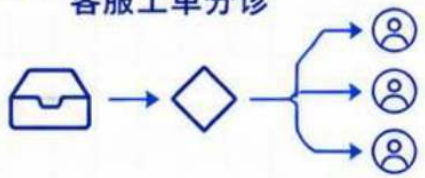
需要可调用 Tool 的模型；先在 .env 配好凭据，再运行场景

一个模板，六种可观察的编排场景

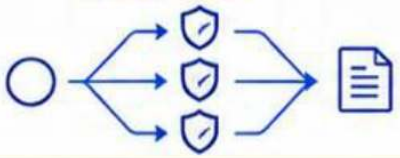
每个场景使用独立 assistant ； 页面只在真实运行证据出现后点亮

选择验证场景

1 Classify and act · 客服工单分诊



2 Fan-out and synthesize · 路由安全检查



3 Adversarial verification · 支付漏洞复核



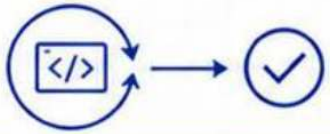
4 Generate and filter · 订单模型优选



5 Tournament · 可读性重构赛



6 Loop until done · 死代码收敛



运行证据

1. **Workflow keyword**
   检测请求中的 workflow
2. **Dynamic triggered**
   观察到解释器 tool call
3. **Agents dispatched**
   来自真实 subagent stream
4. **Workflow complete**
   解释器成功 + 子 Agent 完成 + 最终回答

可展开的运行证据

- { } Generated QuickJS
- >_ Interpreter result

只有 workflow 关键词，不代表已经触发 Dynamic Subagents

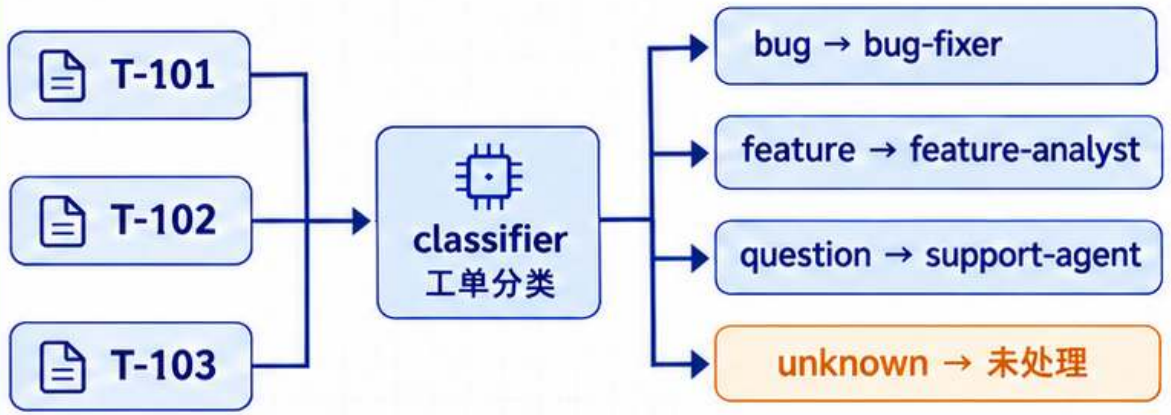
分类后路由：每个分支都保留工单 ID

unknown 不猜；结果保留 id、category 与处理状态。

已配置的子智能体

- name：classifier
  description：工单分类
  system_prompt：不确定返回 unknown
- name：bug-fixer
  description：调查缺陷
  system_prompt：返回复现步骤与影响
- name：feature-analyst
  description：评估功能
  system_prompt：说明价值、条件与取舍
- name：support-agent
  description：回答使用问题
  system_prompt：无依据时明确说明

工单分类与受控路由流程



路由实现（保持 ID、类别与处理结果）

```
async function route(t) {
  const {category} = await classify(t);
  const role = handlers[category];
  if (!role) {
    return {id: t.id, category,
      unhandledReason: "unknown"};
  }
  const result = await task({
    description: `处理 ${t.id}: ${t.text}`,
    subagentType: role
  });
  return {id: t.id, category, result};
}
const results = await Promise.all(tickets.map(route));
```

bug → bug-fixer / feature → feature-analyst / question → support-agent / unknown → 未处理

教学核心片段：classify 用 task + categorySchema；handlers、tickets、Schema 须先定义。验收：三个 ID 都保留；每项只进入一个分支。

场景二：三个文件并行审查，再独立复核

reviewer 先覆盖输入，verifier 再确认或反驳；最多复核 20 条

已配置的子智能体



name: reviewer

description: 审查代码并给证据

system_prompt: 只报告有代码证据的候选



name: verifier

description: 独立复核候选

system_prompt: 先找反证再确认或反驳

建议的 workflow 提示词

运行一个 workflow，审查：

src/routes/login.ts

src/routes/session.ts

src/routes/reset-password.ts

reviewer 返回稳定 ID、文件、行号和证据；

按稳定 ID 去重；verifier 最多复核 20 条。

返回 confirmed，并给出 rejected、unreviewed 数量。

可能生成的 JavaScript / 一种可能写法

```
const files = ["src/routes/login.ts", "src/routes/session.ts",
  "src/routes/reset-password.ts"];
const reviews = await Promise.all(files.map(file => task({
  description: `审查 ${file}; 返回稳定 ID、文件、行号和证据`,
  subagentType: "reviewer", responseSchema: findingsSchema
})));
const unique = [...new Map(reviews.flatMap(r => r.findings)
  .map(x => [x.id, x])).values()];
const sample = unique.slice(0, 20);
const verdicts = await Promies.all(sample.map(x => task({
  description: `复核 ${x.id} @ ${x.file};${x.line}; 证据：${x.evidence}; 先找反证`,
  subagentType: "verifier", responseSchema: verdictSchema
})));
const confirmed = verdicts.filter(v => v.status === "confirmed");
const rejected = verdicts.filter(v => v.status === "rejected");
({ confirmed, rejected, unreviewed: unique.length - sample.length });
```

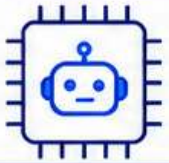
3 个文件 → reviewer → 稳定 ID 去重 → 前 20 条 → verifier → confirmed / rejected / unreviewed

findingsSchema / verdictSchema：自定义对象 Schema，完整代码需先定义

分轮搜索：先去重，再判断是否新增

最多 5 轮；每轮最多保留 20 个新 ID；同批重复也要去掉

核心组件：分析器（analyzer）



```
name: analyzer
description: 分轮查找死代码
system_prompt: 使用稳定 ID，不重复已发现项
```

分轮搜索的 workflow（每轮）



analyzer（分析/查询，得到 items）



按 ID 去重（同批 items 内去重）



排除 seen（过滤已发现的 ID）



截取 20 项（至多保留 20 个新 ID）



空则停止，否则合并（将新 ID 合并到 seen/found）

关键代码（JavaScript）

```javascript
const seen = new Set(), found = [];
let rounds = 0, stopReason = "达到5轮";
for (let r = 0; r < 5; r++) {
  rounds = r + 1;
  const {items} = await analyze([...seen]);
  const unique = [...new Map(
    items.map(x => [x.id, x]))).values()];
  const fresh = unique.filter(
    x => !seen.has(x.id)).slice(0, 20);
  if (!fresh.length) {
    stopReason = "没有新增项"; break;
  }
  for (const x of fresh) {
    seen.add(x.id); found.push(x);
  }
}
```



analyze 为 task 调度辅助函数，须自定义；Schema 约束稳定 ID。5轮/20项是业务规则，不是中间件参数。无新增不代表已证明不存在死代码。

Dynamic Subagents 仍受同一组 Interpreter 参数约束

资源、状态、能力与调试设置继续约束动态编排

一组偏保守的起点

```
CodeInterpreterMiddleware(
  mode="turn",
  subagents=True,
  memory_limit=64 * 1024 * 1024,
  timeout=30.0,
  capture_console=True,
  max_result_chars=8000,
  ptc=["glob"],
  max_ptc_calls=64,
  max_snapshot_bytes=64 * 1024 * 1024,
)
```

资源



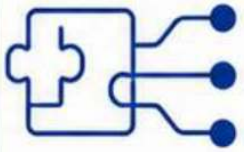
memory_limit ·
timeout ·
max_result_chars

状态



mode ·
max_snapshot_bytes

能力



subagents ·
ptc ·
max_ptc_calls

调试与命名



capture_console ·
tool_name



max_ptc_calls 只限制 tools.*，不限制 task()；
当前没有 max_task_calls



动态调度数量仍要靠输入批次、循环上限、
Prompt 约束和业务预算控制

代码可以不同，流程语义要对

检查约束是否落实，不要求生成代码逐字一致

01 角色存在

subagentType 来自已配置的 name

02 字段正确

description · subagentType · responseSchema?

主 LLM 生成的 JavaScript

03 流程一致

输入逐项覆盖 · 阶段顺序符合提示

04 边界生效

数量上限与停止条件实际进入代码

逐项核对角色、字段、输入覆盖、阶段顺序与停止条件